of failing and returning nothing. The principle behind this type of query is called *fuzziness*, and Elasticsearch employs `fuzzy` queries to forgive spelling mistakes.

Fuzziness is a process of searching for similar terms based on the *Levenshtein distance algorithm* (also referred to as the *edit distance*). The Levenshtein distance is the number of characters that need to be swapped to fetch similar words. For example, searching for "cake" can fetch "take", "bake", "lake", "make", and so on if we use a `fuzzy` query with `fuzziness` (edit distance) set to 1. The following query should return all drama movies because applying a `fuzziness` of `1` to "rama" results in "drama".

Listing 9.22 A `fuzzy` query in action

```
GET movies/_search
{
  "query": {
    "fuzzy": {
      "genre": {
        "value": "rama",
        "fuzziness": 1
      }
    }
  },
  "highlight": {
    "fields": {
      "genre": {}
    }
  }
}
```

In this example, we use the edit distance of 1 (one character) to fetch similar words. You can also try removing a character from the middle of the word, like "dama" or "dram"; these queries also result in positive results when `fuzziness` is set to `1`.

> **NOTE** Unlike a `wildcard` query, which uses a wildcard operator (`*` or `?`), a `fuzzy` query doesn't use operators. Instead, it fetches similar words using the Levenshtein edit distance algorithm.

If we drop one more letter (for example, `"value": "ama"` with `fuzziness` set to `1`), the `fuzzy` query in listing 9.22 does not return any results. Because we are missing two letters, we need to set the edit distance to `2` to solve this problem.

Listing 9.23 A `fuzzy` query with two letters missing in a word

```
GET movies/_search
{
  "query": {
    "fuzzy": {
      "genre": {
        "value": "ama",
```

A word with missing letters